

API 2: Template Mapping

Endpoint: /template-mapping

Method: POST

Purpose:

To **automatically fill a Word document template** (stored on SharePoint) with dynamic data by replacing placeholders, and then **upload the completed document** back to SharePoint.

Conceptual Understanding

Imagine you have a master document template on SharePoint like a **contract agreement**, with placeholders such as:

Dear {cm_customer},

This agreement starts on {cm_startdate} and ends on {cm_enddate}.

The API takes a JSON payload (like your test case) containing real values for these placeholders and produces a **ready-to-use document** — no manual editing needed.

This is extremely useful for automating business workflows like:

- Generating contract agreements
 - Auto-filling purchase orders or invoices
 - Creating personalized reports
-

API #3: RFP Creation

Route

POST /rfp-creation

Purpose

This API **automates the creation of RFP (Request for Proposal) documents** using reference documents and instructions.

It pulls RFP files and answer key documents from SharePoint, processes them using AI

embeddings, and generates a new RFP document **automatically**, all handled in the background.

Think of it as:

👉 “You give me old RFP docs + answer keys + some text instructions → I’ll generate a new, complete proposal document and upload it back to SharePoint.”

📁 Summary Notes (for DSU or Docs)

Aspect	Description
Endpoint	<code>/rfp-creation</code>
Method	<code>POST</code>
Purpose	Automates generation of RFP documents using existing RFPs, answer keys, and given instructions
Key Operations	Downloads templates & answers from SharePoint → creates embeddings → processes AI response → generates <code>.docx</code> → uploads back to SharePoint
Processing	Done asynchronously using background threading
Response	202 Accepted (background process started)
Tech Stack	Flask, threading, Azure OpenAI (embeddings), ChromaDB, MS Graph API (SharePoint integration)
Key Benefit	Saves time by automating manual proposal creation using smart document mapping & AI context

📁 1 rfp_folder_name

➤ Purpose:

This is the **name of the SharePoint folder** where your **RFP template(s)** or **question documents** are stored.

❓ The function `get_specific_files_from_folder()` connects to SharePoint.

❓ It searches for the file(s) in your `rfp_files` list (like "RFP - Airbus to Imperium.docx") **inside the folder named `rfp_folder_name`**.

❓ Then it downloads those files to the local temp folder.

📁 2 answer_folder

► Purpose:

This is the **SharePoint folder containing your company's answer materials or reference documents** — basically, the “knowledge base” the AI will use to generate answers.

► What happens:

- The function fetches **all documents inside that folder**.
- Each file is downloaded and read.
- Then, those files are embedded into the **AI's vector database** (via Azure OpenAI embeddings).
- The AI uses this context to craft relevant responses to the RFP questions.

API 4: Proposal Feedback

Purpose

To **analyze a proposal document** (e.g., a company's bid) and **generate structured feedback** on how well it matches the given RFP requirements.

Endpoint Overview

Property	Description
----------	-------------

URL	/proposal-feedback
-----	--------------------

Method	POST
--------	------

Description Generates feedback on a proposal by comparing it with RFP requirements.

Input Type JSON

Output JSON with AI-generated feedback

API 5: Q&A Document Acceptance

URL: /qna-document-acceptance

Method: POST

1 Purpose — *What It Does and Why It Exists*

This API automates the process of **matching questions and answers** from different documents stored on SharePoint.

Think of it like this:

- You have one document (say “*questions.docx*”) that contains a list of questions from an RFP or tender.
- You have another folder containing **answer key documents** (previously prepared responses, templates, FAQs, etc.).
- The system tries to **find the best matching answer** for each question using a **semantic similarity threshold** (AI-based matching).

Then, instead of directly inserting those answers into a document (like scenario 1 did), it **returns the results as JSON** → so a user can *review, edit, or reject answers* before final acceptance.

 **Goal:** Give user full control over which AI-suggested answers to keep or modify before generating the final populated document.

6 Summary (for your DSU or notes)

Concept	Description
Purpose	Matches Q&A pairs from documents using AI similarity
Input	Question doc, answer folder, and similarity threshold
Processing	Embedding-based similarity using find_answers()
Output	JSON mapping each question → best answer + confidence score
Result Location	Returned in API response (not uploaded to SharePoint)

Concept	Description
Backstory	Built to let users verify AI-generated Q&A matches before document filling
Cleanup	Deletes temp files after each run

API 6

Overview

URL: /populate-answers

Method: POST

Purpose:

Takes the user-reviewed or accepted Q&A pairs (JSON format) and **writes those answers back into the original question document**, then **uploads** the filled document to SharePoint.

So —

- 1 /qna-document-acceptance → compares Q&A docs, finds best answers, returns them.
- 2 User reviews/edits answers manually (front-end step).
- 3 /populate-answers → receives final Q&A pairs, fills them into the question doc, and re-uploads the completed file. 

Folder & File Flow Summary

Stage	Action	Source/Target	Tool Used
1	Download questions_file_name	SharePoint (read drive)	get_sharepoint_url()
2	Process (fill answers)	Temporary local folder	fill_answers_for_questions()
3	Upload final doc	SharePoint (write/upload drive)	upload_to_sharepoint()

Key Points

- This is **final output API** in the Q&A chain.
- **Temp files** are cleaned automatically after upload.
- **If upload_file_name is not given**, system creates:
"originalfilename-populated-answers.docx"
- The **processed document** will appear in your **SharePoint drive configured for uploads** (check your .env READ_DRIVE_ID or WRITE_DRIVE_ID if separated).

API 7

URL: /generate_loss_report

Method: POST

Purpose:

Generates a **loss report (in Markdown format)** for a company — usually related to product or vaccine losses.

This was **developed for Medmard**, so it focuses on pharmaceutical or vaccine-related loss analytics.

High-Level Flow

- 1 **Receives** a company name and document filename (usually product info).
- 2 **Downloads** that .docx file from SharePoint.
- 3 **Extracts** key product/service details from the doc.
- 4 **Runs an AI workflow** using LangGraph to research, summarize, and report product loss info.
- 5 **Returns** the full loss report (as Markdown text) in JSON.

Purpose of /document_tagging API

This API's job is to:

1. **Download** a .docx file from SharePoint.
2. **Analyze** its contents using Azure OpenAI.
3. **Tag** each page or section based on the content (like *Finance*, *Legal*, *Marketing*, etc.).

4. **Add comments** or an **index** inside the document (so users can easily navigate by tag).
 5. **Upload** the tagged document back to SharePoint.
-

Workflow Summary

Here's what happens step-by-step inside the function:

Step	Action	Description
1	Read input data	Takes JSON with filename, upload name, and SharePoint credentials.
2	Create temp files	Makes temporary .docx copies — one for download, one for processed output.
3	Download file	Uses <code>download_sharepoint_file()</code> to fetch from SharePoint → saves locally as <code>temp_doc_path</code> .
4	Tag the file	Uses <code>tag_docx_file()</code> to read document text, call Azure OpenAI, and identify what each page talks about. It writes tags/comments in a new version saved at <code>temp_output_path</code> .
5	Upload new file	Uses <code>upload_to_sharepoint()</code> to push the new tagged version back to the same (or target) drive.
6	Clean up temp files	Deletes local temporary files after upload.
7	Return response	Returns <code>{"message": "File tagged successfully and uploaded to SharePoint."}</code> .

API Overview — /map_pricing_table

Purpose

To **automatically fill (map) pricing data** into a Word document's table template.

This API:

1. **Downloads** a DOCX template (usually containing placeholders like {name}, {price}, {country}) from SharePoint.
2. **Replaces** the placeholders in a pricing table with actual data (provided in JSON).
3. **Uploads** the completed document back to SharePoint.

It's part of the Finance & Operations automation pipeline for Medmard.

End-to-End Flow

Step	Action	Description
1	Receive Request (JSON)	Includes file name, pricing data, and SharePoint credentials.
2	Download DOCX	Uses <code>get_sharepoint_url()</code> + <code>requests.get()</code> to fetch the document from SharePoint.
3	Create Temp Paths	Saves the downloaded document to a temporary file and prepares an output path.
4	Map Data	Calls <code>pricing_table()</code> to replace placeholders {} with real pricing values.
5	Upload Back	Uses <code>upload_to_sharepoint()</code> to upload the filled version.
6	Clean Up	Deletes all temporary files and folders.

Feature / Behavior	v1	v2	v3
Validation of inputs	✗ Minimal validation	✓ Proper validation of required SharePoint & OData params	⚠ Minimal validation (not as strict as v2)
Purpose	Replace non-matching clauses with standard ones	Identify and categorize clauses (no replacements)	Replace or highlight clauses (user-configurable)
State fields	all_changes → stores replacement details	all_identifications → stores clause identification info	all_changes again, but enhanced with replace & highlight flags
Customization options	None	None	✓ Adds replace and highlight flags for flexible behavior
LLM-powered summary	✗ None	✗ None	✓ Generates AI summary of all contract changes using LLM
Outputs	Detailed JSON list of changes	JSON list of identified clause categories	JSON list of changes + AI textual summary
Use case	Automated clause replacement	Clause detection / auditing	Clause replacement + summarization (hybrid)
Upload back to SharePoint	✓ Yes	✓ Yes	✓ Yes
Code readability	Simple	Better structured	Most advanced but longer due to LLM handling